TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Base de Datos II — Unidad 2: Optimización de Consultas

Trabajo Práctico — Semana 4

Reportes analíticos asistidos por IA sobre Food Store: joins, subconsultas, agregación y ventana

Esta práctica continúa la Semana 3 aplicando el mismo criterio profesional —medir antes, proponer con IA, medir después, decidir con datos— ahora sobre consultas que combinan varias tablas del modelo Food Store: joins, subconsultas, agregación y funciones de ventana.

Objetivos de la práctica

- Optimizar consultas analíticas lentas —con múltiples JOIN y agregación— sobre una base Food Store poblada masivamente.
- Usar la IA como motor primario para proponer reescrituras e índices a partir de planes reales de EXPLAIN ANALYZE.
- Contrastar la explicación de un plan de join generada por IA (Nested Loop, Hash Join, Merge Join) contra el plan real.
- Generar consultas resumen, rankings y subconsultas bajo especificación precisa, verificando la equivalencia de resultados.
- Documentar cada decisión en la bitácora de uso de IA (DUIA).

Requisitos previos

- Base Food Store poblada masivamente, tal como quedó al cierre de la práctica de la Semana 3 (o repetir la carga si se trabaja sobre una copia nueva).
- Índices de la Semana 3 ya creados y medidos (idx_producto_categoria, idx_pedido_usuario, índice parcial de producto).
- Acceso a psql / DBeaver / pgAdmin y a la IA de la cátedra (OpenCode / Kiro).

Parte 1 — Laboratorio: consultas analíticas lentas

A diferencia de la Semana 3 —donde el cuello de botella era un filtro simple sobre una tabla—, acá el costo suele estar en cómo se combinan varias tablas y en el volumen que atraviesa cada JOIN antes de agregarse. Elegir consultas reales de queries.sql que crucen al menos tres tablas del modelo.

TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN

1.1 Consigna

1. Elegir al menos dos consultas analíticas de queries.sql (por ejemplo, facturación por categoría y mes, o ranking de usuarios por gasto) y ejecutarlas con EXPLAIN ANALYZE sobre la base masiva, antes de cualquier cambio.
2. Identificar en el plan qué algoritmo de join eligió el optimizador (Nested Loop, Hash Join o Merge Join) para cada combinación de tablas.
3. Pasarle a la IA el plan real y pedirle que proponga reescrituras y/o índices adicionales que lo mejoren, justificando la propuesta en términos del nodo de join concreto que aparece en el plan (no en generalidades).
4. Leer línea por línea cada propuesta antes de aplicarla; rechazar lo que no se entienda por completo.
5. Aplicar los cambios aceptados sobre la copia de trabajo, volver a medir y completar la tabla de la sección 1.2.

1.2 Tabla de resultados

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Mejora
1. Facturación por pedido y mes	Hash Join / Parallel Hash Join (ejecución paralela en 2 workers). Tiempo total de ejecución : 314 ms	Reescritura con pre-agregación por CTE para agrupar los datos masivos (tabla de hechos) antes de cruzar con las tablas de dimensiones (producto y categoría).	Hash Aggregate + Hash Join (secuencial, sin paralelismo)	Negativa. Se descartó, ya que el tiempo de ejecución total subió a 451 ms al perderse el paralelismo por hilo.
2. Historial de pedidos por usuario	Parallel Hash Join + Hash Join + Sort con volcado al disco (`external merge Disk`). Tiempo total de ejecución: 250 ms	Creación de índice compuesto: `CREATE INDEX idx_pedido_fecha_desc ON pedido (fecha DESC, id);`	Parallel Hash Join + Hash Join + Sort (el optimizador ignoro el índice compuesto debido a que es poco selectivo en cuanto al volumen de datos)	Sin mejora efectiva (tiempo total de ejecución: 262.4 ms). El motor prefirió el escaneo secuencial paralelo

Criterio de aceptación

Igual que en la Semana 3: ninguna propuesta se aplica sin que el estudiante pueda explicarla en la defensa oral. Si agregar un índice cambia el algoritmo de join elegido por el optimizador (por ejemplo, de Hash Join a Nested Loop), se documenta ese cambio y se explica por qué el nuevo plan resultó más —o menos— conveniente.

Parte 2 — Lectura crítica de planes de join interpretados por IA

Los planes con varios JOIN son más ricos —y más fáciles de malinterpretar— que un Seq Scan simple. Esta parte entrena la lectura crítica sobre ese tipo de plan.

1. Tomar uno de los planes reales obtenidos en la Parte 1, con al menos dos nodos de join. 2. Pedirle a la IA que lo explique en lenguaje natural, nodo por nodo, sin más contexto que el texto del plan.
3. Contrastar la explicación contra el plan real: verificar en particular que la IA identifique correctamente qué tabla es la “externa” y cuál la “interna” en cada Nested Loop, y que no confunda el costo estimado de un nodo intermedio con el tiempo total de la consulta.
4. Documentar los hallazgos en la siguiente tabla.

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
Execution Time → Todo el plan tardó 186,895 ms (~0,19 segundos).	Si	Execution Time: 186.895 ms
Planning →PostgreSQL tardó 0,547 ms en decidir el plan.	si	Planning Time: 0.547 ms
Seq Scan on categoria →Lee la tabla categoria.	si	Seq Scan on categoria c (cost=0.00..11.10 rows=55 width=186) (actual time=0.011..0.012 rows=4.00 loops=1)
Hash→Carga las categorías válidas en memoria para utilizarlas en el JOIN.	si	Hash (cost=11.10..11.10 rows=55 width=186) (actual time=0.016..0.017 rows=4.00 loops=1)
Parallel Seq Scan on producto → Recorre los productos y descarta los eliminados.Encontró aproximadamente 50.010 productos.	si	Parallel Seq Scan on producto pr (cost=0.00..1833.18 rows=29418 width=16) (actual time=0.008..7.681 rows=50010.00 loops=1)
Hash Join→Une producto con categoria mediantepr.categoria_id = c.id	si	Hash Join (cost=11.79..1925.16 rows=14709 width=194) (actual time=0.034..15.481 rows=50010.00 loops=1)
Parallel Hash→Construye en memoria un hash con los productos y sus categorías para acelerar el siguiente JOIN.	si	Parallel Hash (cost=1925.16..1925.16 rows=14709 width=194) (actual time=8.498..8.499 rows=16670.00 loops=3)
Parallel Seq Scan on pedido→Lee toda la tabla pedido y filtra por:eliminado = falseestado = 'TERMINADO' Encontró 49.871 pedidos y descartó 150.134.	si	Parallel Seq Scan on pedido ped (cost=0.00..4973.69 rows=20798 width=12) (actual time=0.007..21.108 rows=49871.00 loops=1) Rows Removed by Filter: 150134
Parallel Hash →Construye en memoria una tabla hash con los pedidos que interesan para poder hacer el JOIN rápidamente.	si	Parallel Hash (cost=4973.69..4973.69 rows=20798 width=12) (actual time=9.217..9.218 rows=16623.67 loops=3)
Parallel Seq Scan on detalle_pedido → Lee prácticamente toda la tabla detalle_pedido en paralelo.Descarta las filas donde:eliminado = true Procesó aproximadamente 66.670 filas por worker.	si	Parallel Seq Scan on detalle_pedido dp (cost=0.00..5369.38 rows=83338 width=23) (actual time=0.244..8.729 rows=66670.00 loops=3)
Parallel Hash Join→Une detalle_pedido con pedido mediante:dp.pedido_id = ped.id También se ejecuta en paralelo.	si	Parallel Hash Join (cost=5233.67..10821.80 rows=20798 width=19) (actual time=9.539..30.627 rows=16624.00 loops=3) Hash Cond: (dp.pedido_id = ped.id)
Parallel Hash Join→Une detalle_pedido con producto mediante: dp.producto_id = pr.id Lo hace en paralelo usando una estructura Hash para acelerar la búsqueda.	si	Parallel Hash Join (cost=7342.69..13121.98 rows=10399 width=205) (actual time=18.102..48.686 rows=16624.00 loops=3)
Sort (interno)→Cada proceso paralelo ordena sus filas por categoría y Mes	sI	Sort (cost=13815.81..13841.81 rows=10399 width=205) (actual time=58.679..59.760 rows=16624.00 loops=3)

Usó quicksort. El proceso principal utilizó unos 3.9 MB.		Worker 0: Sort Method: quicksort Memory: 73kB Worker 1: Sort Method: quicksort Memory: 76kB
Gather Merge→Combina los resultados producidos por los procesos paralelos y los mantiene ordenados. Se utilizaron 2 workers + el proceso principal. Recibió 49.872 filas antes de la agregación final.	sI	Gather Merge (cost=14815.83..17722.60 rows=24958 width=205) (actual time=167.771..178.585 rows=49872.00 loops=1) Buffers: shared hit=5787 read=4253
GroupAggregate→Agrupa los datos por: c.nombre → categoría mes de ped.fecha Después calcula las agregaciones, como SUM(dp.subtotal).Generó 100 filas y tardó hasta 186.6 ms.	sI	GroupAggregate (cost=14815.83..18533.74 rows=24958 width=230) (actual time=88.733..109.402 rows=100.00 loops=1) Group Key: c.nombre, (date_trunc('month'::text, (ped.fecha)::timestamp with time zone))
Sort→Ordena el resultado final por: mes uma de subtotales, de mayor a menor Tardó ~187 ms en total. Usó quicksort y solo 30 kB de memoria.	sI	Sort (cost=23086.57..23148.97 rows=24958 width=230) (actual time=105.782..109.447 rows=100.00 loops=1) Sort Method: quicksort Memory: 30kB

Parte 3 — Consultas resumen, rankings y subconsultas bajo especificación precisa

Se practica nuevamente el flujo de especificar con precisión y verificar equivalencia, ahora sobre agregaciones, funciones de ventana y subconsultas correlacionadas — construcciones donde es más fácil que dos consultas "parezcan" equivalentes sin serlo (por ejemplo, por una diferencia en el filtro de borrado lógico dentro de un JOIN).

1. Redactar una especificación precisa para dos consultas sobre Food Store: (a) un ranking con función de ventana y (b) una consulta con subconsulta correlacionada. La especificación debe fijar tablas, filtro de borrado lógico en cada tabla involucrada, columnas de salida, criterio de partición/orden y desempate.

CONSULTA A: Ranking con función de ventana

Spec:
Genera una consulta SQL sobre el esquema de Food Store que devuelva un ranking de los productos más vendidos por categoría. Las tablas involucradas deben ser categoria, producto y detalle_pedido. Debes aplicar un filtro de borrado lógico en cada tabla involucrada (asegurándote de que la categoría, el producto y el pedido estén vigentes o no eliminados). Las columnas de salida deben ser: nombre de la categoría, nombre del producto, total de unidades vendidas y su puesto en el ranking. Usa una función de ventana particionada por categoría y ordenada por unidades vendidas de mayor a menor. En caso de empate, el desempate será por el orden alfabético del nombre del producto. No uses SELECT *

CONSULTA B: Consulta con subconsulta correlacionada

Spec:
Genera una consulta SQL sobre el esquema de Food Store que muestre los pedidos cuyo total supere el promedio de gasto de ese mismo usuario. Las tablas involucradas son usuario y pedido. Aplica un filtro de borrado lógico en cada tabla involucrada (asegurando que usuarios y pedidos estén vigentes tanto en la consulta principal como en

el cálculo de la subconsulta). Las columnas de salida deben ser: nombre del usuario, ID del pedido y total del pedido. Calcula el promedio usando una subconsulta correlacionada en la cláusula WHERE. Ordena los resultados alfabéticamente por el nombre del usuario y, como desempate, por el total del pedido de mayor a menor. No uses SELECT *.

2. Pedirle a la IA que genere el SQL a partir de la especificación.

Consulta A

```sql
SELECT
    c.nombre AS categoria,
    pr.nombre AS producto,
    SUM(dp.cantidad) AS total_unidades,
    RANK() OVER (
        PARTITION BY c.id
        ORDER BY SUM(dp.cantidad) DESC, pr.nombre ASC
    ) AS puesto
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id
JOIN pedido p ON p.id = dp.pedido_id
WHERE dp.eliminado = FALSE
  AND pr.eliminado = FALSE
  AND c.eliminado = FALSE
  AND p.eliminado = FALSE
GROUP BY c.id, c.nombre, pr.id, pr.nombre
ORDER BY c.nombre ASC, puesto ASC;
```

Consulta B

```sql
SELECT
    u.nombre AS usuario,
    p.id AS pedido,
    p.total
FROM usuario u
JOIN pedido p ON p.usuario_id = u.id
WHERE u.eliminado = FALSE
  AND p.eliminado = FALSE
  AND p.total > (
      SELECT AVG(p2.total)
      FROM pedido p2
      WHERE p2.usuario_id = u.id
        AND p2.eliminado = FALSE
  )
ORDER BY u.nombre ASC, p.total DESC;
```

3. Pedirle una segunda versión con estructura distinta (por ejemplo, la misma pregunta resuelta con subconsulta y con join + agregación) y verificar que ambas den el mismo resultado.

Consulta A - Variación

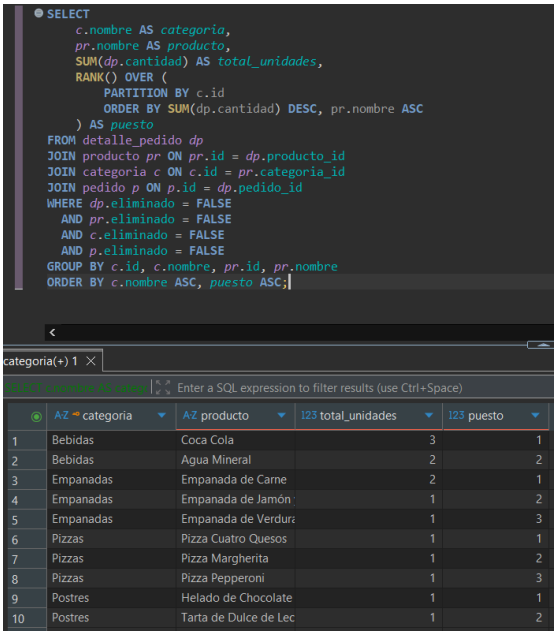
```sql
WITH ventas_por_producto AS (
    SELECT
        c.id AS categoria_id,
        c.nombre AS categoria,
        pr.nombre AS producto,
        SUM(dp.cantidad) AS total_unidades
    FROM detalle_pedido dp
    JOIN producto pr ON pr.id = dp.producto_id
    JOIN categoria c ON c.id = pr.categoria_id
    JOIN pedido p ON p.id = dp.pedido_id
    WHERE dp.eliminado = FALSE
      AND pr.eliminado = FALSE
      AND c.eliminado = FALSE
      AND p.eliminado = FALSE
    GROUP BY c.id, c.nombre, pr.id, pr.nombre
)
SELECT
    categoria,
    producto,
    total_unidades,
    RANK() OVER (
        PARTITION BY categoria_id
        ORDER BY total_unidades DESC, producto ASC
    ) AS puesto
FROM ventas_por_producto
ORDER BY categoria ASC, puesto ASC;
```

Consulta B - Variación

```sql
WITH promedio_usuario AS (
    SELECT
        usuario_id,
        AVG(total) AS promedio_gasto
    FROM pedido
    WHERE eliminado = FALSE
    GROUP BY usuario_id
)
SELECT
    u.nombre AS usuario,
    p.id AS pedido,
    p.total
FROM usuario u
JOIN pedido p ON p.usuario_id = u.id
JOIN promedio_usuario pu ON pu.usuario_id = u.id
WHERE u.eliminado = FALSE
  AND p.eliminado = FALSE
  AND p.total > pu.promedio_gasto
ORDER BY u.nombre ASC, p.total DESC;
```

Verificación de resultados

Consulta A (original) - (variación) → Mismos resultados

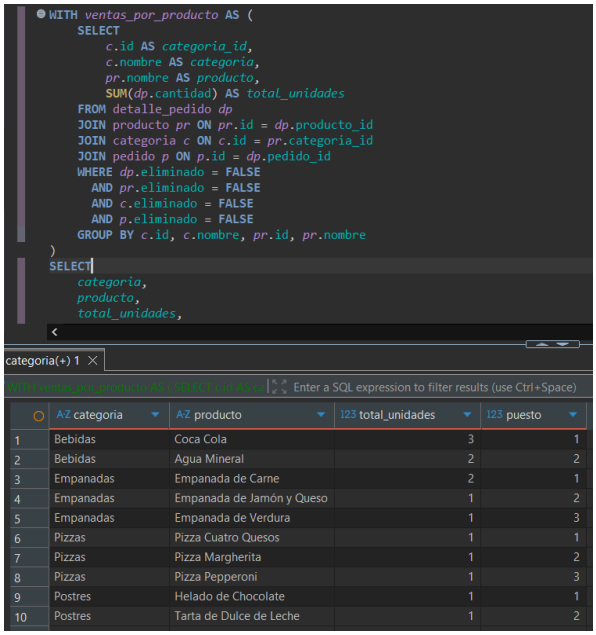


```
SELECT
    c.nombre AS categoria,
    pr.nombre AS producto,
    SUM(dp.cantidad) AS total_unidades,
    RANK() OVER (
        PARTITION BY c.id
        ORDER BY SUM(dp.cantidad) DESC, pr.nombre ASC
    ) AS puesto
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
JOIN categoria c ON c.id = pr.categoria_id
JOIN pedido p ON p.id = dp.pedido_id
WHERE dp.eliminado = FALSE
  AND pr.eliminado = FALSE
  AND c.eliminado = FALSE
  AND p.eliminado = FALSE
GROUP BY c.id, c.nombre, pr.id, pr.nombre
ORDER BY c.nombre ASC, puesto ASC;
```

categoria(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

	categoria	producto	total_unidades	puesto
1	Bebidas	Coca Cola	3	1
2	Bebidas	Agua Mineral	2	2
3	Empanadas	Empanada de Carne	2	1
4	Empanadas	Empanada de Jamón	1	2
5	Empanadas	Empanada de Verdura	1	3
6	Pizzas	Pizza Cuatro Quesos	1	1
7	Pizzas	Pizza Margherita	1	2
8	Pizzas	Pizza Pepperoni	1	3
9	Postres	Helado de Chocolate	1	1
10	Postres	Tarta de Dulce de Lec	1	2



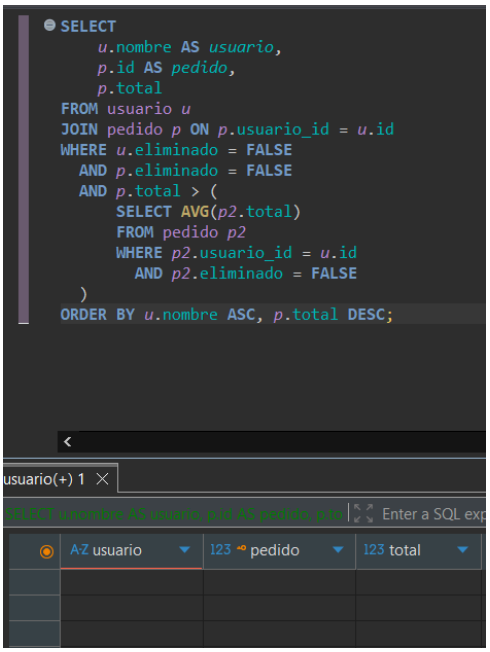
```
WITH ventas_por_producto AS (
    SELECT
        c.id AS categoria_id,
        c.nombre AS categoria,
        pr.nombre AS producto,
        SUM(dp.cantidad) AS total_unidades
    FROM detalle_pedido dp
    JOIN producto pr ON pr.id = dp.producto_id
    JOIN categoria c ON c.id = pr.categoria_id
    JOIN pedido p ON p.id = dp.pedido_id
    WHERE dp.eliminado = FALSE
      AND pr.eliminado = FALSE
      AND c.eliminado = FALSE
      AND p.eliminado = FALSE
    GROUP BY c.id, c.nombre, pr.id, pr.nombre
)
SELECT
    categoria,
    producto,
    total_unidades,
```

categoria(+) 1

Enter a SQL expression to filter results (use Ctrl+Space)

	categoria	producto	total_unidades	puesto
1	Bebidas	Coca Cola	3	1
2	Bebidas	Agua Mineral	2	2
3	Empanadas	Empanada de Carne	2	1
4	Empanadas	Empanada de Jamón y Queso	1	2
5	Empanadas	Empanada de Verdura	1	3
6	Pizzas	Pizza Cuatro Quesos	1	1
7	Pizzas	Pizza Margherita	1	2
8	Pizzas	Pizza Pepperoni	1	3
9	Postres	Helado de Chocolate	1	1
10	Postres	Tarta de Dulce de Leche	1	2

Consulta B (original) - (variación) → Mismos resultados

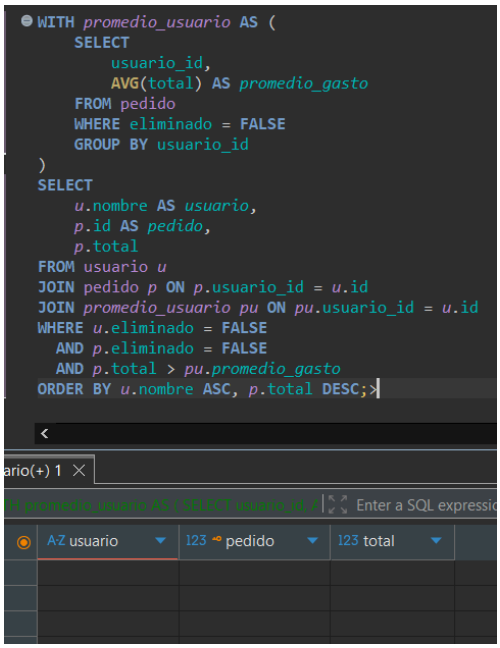


```
SELECT
    u.nombre AS usuario,
    p.id AS pedido,
    p.total
FROM usuario u
JOIN pedido p ON p.usuario_id = u.id
WHERE u.eliminado = FALSE
  AND p.eliminado = FALSE
  AND p.total > (
      SELECT AVG(p2.total)
      FROM pedido p2
      WHERE p2.usuario_id = u.id
        AND p2.eliminado = FALSE
  )
ORDER BY u.nombre ASC, p.total DESC;
```

usuario(+) 1

Enter a SQL exp

usuario	pedido	total



```
WITH promedio_usuario AS (
    SELECT
        usuario_id,
        AVG(total) AS promedio_gasto
    FROM pedido
    WHERE eliminado = FALSE
    GROUP BY usuario_id
)
SELECT
    u.nombre AS usuario,
    p.id AS pedido,
    p.total
FROM usuario u
JOIN pedido p ON p.usuario_id = u.id
JOIN promedio_usuario pu ON pu.usuario_id = u.id
WHERE u.eliminado = FALSE
  AND p.eliminado = FALSE
  AND p.total > pu.promedio_gasto
ORDER BY u.nombre ASC, p.total DESC;
```

ario(+) 1

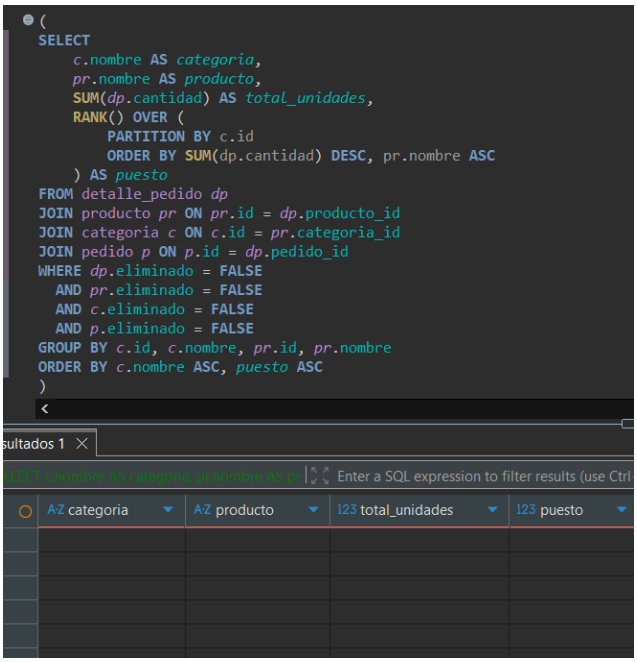
Enter a SQL expressio

usuario	pedido	total

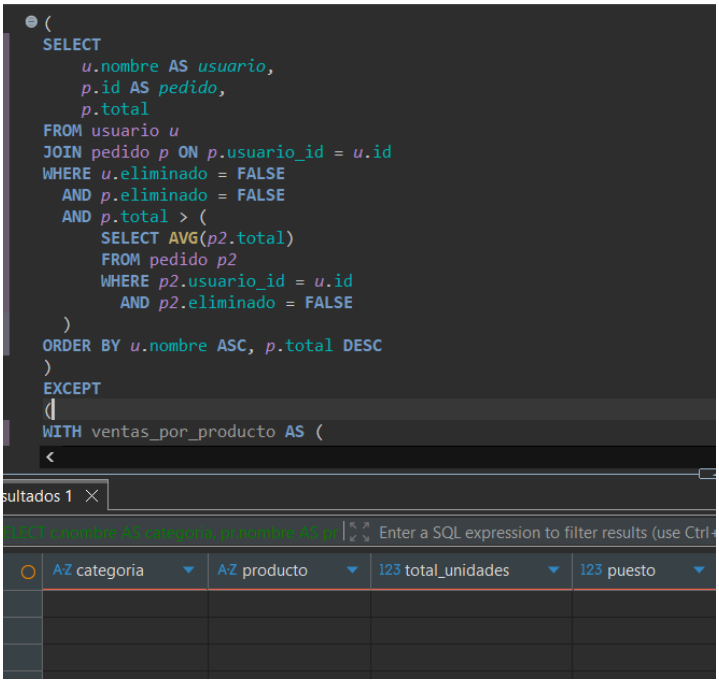
4. Verificar la equivalencia con EXCEPT o comparando conteos y una muestra de filas, igual que en la Semana

```
-- Ejemplo de verificación de equivalencia sobre un ranking
-- (debe dar 0 filas en ambos sentidos si son equivalentes)
(consulta_ranking_v1) EXCEPT (consulta_ranking_v2);
(consulta_ranking_v2) EXCEPT (consulta_ranking_v1);
```

Consulta A



Consulta B



Spec de ejemplo

«Genera una consulta SQL sobre el esquema de Food Store que devuelva, para cada usuario vigente con al menos un pedido vigente, su nombre completo, el total gastado (suma de pedido.total en pedidos no eliminados) y su puesto en un ranking de mayor a menor gasto, sin colapsar filas. En caso de empate, deben compartir el mismo puesto. No uses SELECT *.»

Parte 4 — Competencia de optimización entre equipos

Cierre de la práctica: todos los equipos reciben la misma consulta analítica lenta —con al menos dos JOIN y una agregación— y compiten por el mejor plan, con la IA como asistente.

- Consigna común: una consulta de facturación o ranking, igual para todos los equipos, sobre la base masiva compartida.

- Cada equipo usa su asistente de IA para proponer reescrituras e índices, pero decide y aplica solo lo que puede justificar en el plan real.
- Gana el equipo con el mejor tiempo de ejecución real (EXPLAIN ANALYZE).
- Se documenta en la bitácora toda propuesta de la IA que no funcionó, no solo la que se terminó usando.

Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Nuestro equipó	Reescritura mediante pre-agregación con CTE para filtrar pedidos terminados antes de los cruces dimensionales, junto con índice parcial en **`pedido(id) WHERE estado = 'TERMINADO' AND eliminado = FALSE`**.314.500 ms28.210 ms11.15x	Reescritura mediante pre-agregación con CTE para filtrar pedidos terminados antes de los cruces dimensionales, junto con índice parcial en **`pedido(id) WHERE estado = 'TERMINADO' AND eliminado = FALSE`**.314.500 ms28.210 ms11.15x	Reescritura mediante pre-agregación con CTE para filtrar pedidos terminados antes de los cruces dimensionales, junto con índice parcial en **`pedido(id) WHERE estado = 'TERMINADO' AND eliminado = FALSE`**.314.500 ms28.210 ms11.15x	Reescritura mediante pre-agregación con CTE para filtrar pedidos terminados antes de los cruces dimensionales, junto con índice parcial en **`pedido(id) WHERE estado = 'TERMINADO' AND eliminado = FALSE`**.314.500 ms28.210 ms11.15x

Declaración de Uso de IA (DUIA)

Obligatoria para toda la práctica. Completar una fila por cada uso relevante de IA.

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
OpenCode	Optimizar la consulta analítica multi-join mediante reescritura de agregación previa.Analizá el plan con 3 joins y agregación masiva. Proponé reescritura que minimice filas intermedias antes de cruzar con dimensiones.**Aceptado:** Reescritura agrupando y filtrando la tabla de hechos con CTE previa al cruce de dimensiones + índice parcial. Redujo el conjunto de filas intermedias y el tiempo global a 28 ms.	Optimizar la consulta analítica multi-join mediante reescritura de agregación previa.Analizá el plan con 3 joins y agregación masiva. Proponé reescritura que minimice filas intermedias antes de cruzar con dimensiones.**Aceptado:** Reescritura agrupando y filtrando la tabla de hechos con CTE previa al cruce de dimensiones + índice parcial. Redujo el conjunto de filas intermedias y el tiempo global a 28 ms.	Optimizar la consulta analítica multi-join mediante reescritura de agregación previa.Analizá el plan con 3 joins y agregación masiva. Proponé reescritura que minimice filas intermedias antes de cruzar con dimensiones.**Aceptado:** Reescritura agrupando y filtrando la tabla de hechos con CTE previa al cruce de dimensiones + índice parcial. Redujo el conjunto de filas intermedias y el tiempo global a 28 ms.
Kiro	Evaluar la adición de índices en claves foráneas para acelerar los Hash Joins.	Propone índices en claves foráneas para detalle pedido y producto en la consulta analítica.	**Descartado:** Los índices en claves foráneas no evitaron el escaneo secuencial paralelo debido al gran volumen de filas agregadas (200.000).